

雲端分散式計算系統 · 個人功能報告

商場購物搶購 多代理人路徑規劃

Multi-Agent Path Finding Simulation (PBS + A*)

學生：【填入姓名】 | 學號：【填入學號】

日期：2026-06-10

Docker Compose

Python / Flask

SQLite WAL

PBS MAPF

四容器分散式架構

以 Docker Compose 編排，共享 SQLite 資料庫

容器職責

web

Apache + mod_wsgi + Flask
API、Dispatcher、靜態前端

node1 / node2 / node3

Gunicorn (1 worker) + Flask
執行 PBS 模擬，回傳 GIF

共用資源

- SQLite volume (WAL mode)
- REST callback 介面



排程、心跳與容錯機制

Job 狀態機

pending

↓ dispatch

dispatched

↓ /run 成功

running

↓ callback

done / cancelled

Dispatcher

- 找 idle 節點 + pending job
- **BEGIN IMMEDIATE** 交易鎖，防止搶單
- 背景 scanner 每 5s 掃描
- 節點 offline → job 重排為 pending
- 節點重啟後孤立 running job → 重排

Heartbeat

- 節點每 5 秒回報一次
- 帶 CPU% 與 MEM%
- 超過 15 秒未回報 → offline
- Web 節點卡顯示 → scanner 自動恢復

SQLite WAL Mode — 為什麼夠用？

- Write-Ahead Log：寫入不鎖整個檔案
- 多個讀取者可同時讀（無讀讀鎖）
- Dispatcher 用 **BEGIN IMMEDIATE** 確保寫入原子性
- 3 節點並發量小，WAL 性能完全足夠

購物搶購 MAPF 模擬系統

Multi-Agent Path Finding — 多人在商場裡同時購物，互相避讓

問題情境

- 地圖：牆 / 貨架 / 收銀台 / 入口
- 每位顧客 (agent) 有隨機購物清單
- 多人同時行動，不能佔同一格
- 目標：所有人完成購物並結帳

系統功能

- Canvas 地圖編輯器 (拖曳繪製)
- 4 種方案預設 (20×20 ~ 60×60)
- 算法進階參數即時調整
- GIF 動畫 + 詳細 stats 面板
- 節點 CPU/MEM 即時監控

【放 Canvas 編輯器截圖】
地圖編輯介面

【放 GIF 動畫截圖 (幾個關鍵幀) 】
模擬結果動畫

PBS — Priority-Based Search

固定優先序、逐層規劃的多代理人路徑搜索

MAPF 衝突種類

頂點衝突：兩人在同一時間佔同一格

✓ PBS 完全避免

交換衝突（對穿）：兩人互換相鄰格

✓ 本專案已修正 (`edge_reserved`)

PBS 規劃流程

1. 依 agent 索引排定固定優先序
2. 高優先 agent 先執行 Time-Expanded A*
3. 將其路徑加入 `reserved` 集合
4. 低優先 agent 的 A* 必須繞開 `reserved`
5. 每個模擬步驟重新規劃一次

算法特性

優點

- 實作簡單，速度快
- 頂點衝突完全消除
- 適合教學展示

限制

- 不保證全域最優解
- 低優先 agent 可能顯著變慢
- 固定優先序對 agent 0 有利

更優算法：CBS (最優解) 或 WHCA* (滑動視窗重規劃)

本系統已規劃 WHCA* 升級路線

Time-Expanded A* 與六項改進

Time-Expanded A*

搜索空間：(r, c, t) 三維

每個狀態為「在 t 時刻位於格子 (r,c)」

允許等待（原地停留）

max_t 限制搜索深度

- 障礙：WALL + 非目標的 SHELF/CASHIER
- reserved 集合：高優先 agent 的 (r,c,t)
- Heuristic：Manhattan 距離
- 回傳 None → agent 原地等待此步

關鍵參數

- goal_reserve：抵達目標後保留格子步數
- min_t_floor：A* 最短時間視窗
- spawn_interval：每批進場間隔步數

本次實作的六項改進

- 1 **BFS 可達性預檢**：模擬前驗證所有貨架/收銀台從入口可達，不可達立即報錯
- 2 **goalReserve 自動上限**：clamp 至 $\text{max_steps} \div 4$ ，防止收銀台永久死鎖
- 3 **貪婪最近鄰清單排序**：從入口出發，依距離重排購物順序，減少來回折返
- 4 **對穿衝突修正**：edge_reserved 記錄每個移動的反向邊，A* 展開時禁止互穿
- 5 **動態收銀台選擇**：購物完成時才選收銀台，分數 = 距離 + 排隊人數 $\times$ goal_reserve
- 6 **入口擁塞重試**：等待逾 $\text{spawn_interval} \times 2$ 步後自動改走備用入口

系統操作流程與動畫展示

① 選方案 → ② 調參數 → ③ 提交 → ④ 輪詢 → ⑤ 看結果

- Canvas 右鍵清除格子，拖曳批次繪製
- 進階 ▾ 展開後可調整算法參數
- 任務佇列每 3s 自動輪詢狀態
- 完成後點「查看資訊」展開購物記錄

【節點監控截圖】
CPU / MEM 顯示

【放 GIF 動畫截圖 — 起始幀】
小小超市 20×20，agents=3
顯示入口、貨架、收銀台標籤

【放 GIF 動畫截圖 — 中間/結束幀】
agents 走向各自收銀台
GIF 右側圖例欄

各方案參數與結果

方案	地圖	Agents	清單長度	步數上限	goal_reserve	典型完成率
小小超市	20×20	3	2	80	10	【填入】
小超市	60×60	10	3	300	75*	【填入】
中型商場	60×60	20	4	500	125*	【填入】
大型賣場	60×60	35	5	800	200*	【填入】

* goal_reserve 自動 clamp 至 max_steps÷4

關鍵指標說明

- makespan：最後一位完成的步數 (越小越好)
- sum_of_costs：所有 agent 完成步數總和
- agents_done：在步數限制內完成的人數
- 超時的 agent 計為 max_steps (懲罰分)

【放 stats 面板截圖】
含購物清單 / 收銀台分配 / 售罄記錄

常見問題整理

Q1. PBS 為什麼會超時？

低優先 agent 必須繞開所有人的保留路徑，人數越多可用路徑越少。goal_reserve 過高時，收銀台被鎖太久，後到者 A* 看不到空位，只能停在原地。

Q2. 節點掛掉工作會怎樣？

Heartbeat 超時後 background_scanner 將對應節點標為 offline，該節點的 running job 重排為 pending，由其他節點接手。

Q3. SQLite 能支援多節點並發嗎？

WAL mode 允許多讀者同時讀，Dispatcher 用 BEGIN IMMEDIATE 確保同一 job 不會被兩個節點同時搶走。3 節點量級完全足夠。

Q4. 為什麼選 PBS 不是 CBS？

PBS 實作簡單、速度快，適合教學展示。CBS 可保證最優解但複雜度顯著更高。本系統已規劃 WHCA* 作為升級路線。

Q5. WHCA* 和 PBS 的差別？

WHCA* 以滑動視窗 (window) 重規劃，所有 agent 在每個視窗內平等競爭，解決 PBS 固定優先序對 agent 0 過度有利的問題，公平性更好。

Q6. 系統可以水平擴展嗎？

可以，新增 node4 只需在 docker-compose.yml 複製一個 node 服務並設定 SELF_ID 環境變數，web 的 Dispatcher 發現新節點 heartbeat 後會自動排程。

延伸議題、遇到的問題與 GitHub

延伸議題

WHCA* 算法升級

滑動視窗重規劃，改善固定優先序的公平性問題

購物路線 TSP 最佳化

目前已用貪婪最近鄰，進一步可用 2-opt 改進

多樓層地圖 / 電梯節點

擴展地圖結構，加入垂直移動節點

遇到的問題與解法

- goalReserve 造成收銀台死鎖
→ 自動 clamp 至 $\text{max_steps} \div 4$
- 節點重啟後 running job 永遠卡住
→ scanner 補上 idle-node running job 重排邏輯
- 前端 Canvas 多尺寸適配
→ ROWS/COLS 改為動態變數，切換方案時自動調整畫布
- 瀏覽器快取導致 JS 更新未生效
→ 無痕模式或強制刷新 app.js

GitHub 專案連結

【填入 GitHub URL】